

Dimension Reduction Techniques

Principal Component Analysis and Factor Analysis

Atanu Kumar Ghosh
Presidency University

Workshop on Statistics and Machine Learning in Practice
In the memory of Late Professor Dwijesh Dutta Majumdar
Organised by
Department of Statistics with IQAC, Brahmanada Kesab Chandra College
In Collaboration with
Department of Statistics, West Bengal State University

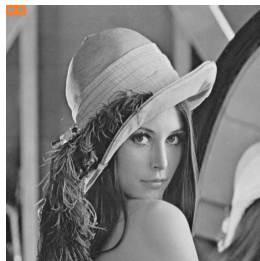
July 27, 2020

High Dimensional Paradigm

- Most of the modern day applications produce data that are high dimensional.

High Dimensional Paradigm

- Most of the modern day applications produce data that are high dimensional.
- A 1024×1024 gray scale image contain 1048576 pixels and each pixel having the gray scale value.



- Can be considered as a 1048576 component vector or a data point in $\mathbb{R}^{1048576}$.
- For medical images and massive astrophysics images, the number p of pixels can be larger than the number n of images.

Curse of dimensionality

- Being able to sense too many variables often causes a problem which we called **curse of dimensionality**.
 - cannot apply ordinary least square estimates
 - problem in local regression techniques, like K-NN
 - accumulation of errors

Intrinsic low dimensional structure

- A fact of hope is that observations are not randomly scattered in $\mathbb{R}^d$, rather they are concentrated around some low dimensional subspaces.

Intrinsic low dimensional structure

- A fact of hope is that observations are not randomly scattered in $\mathbb{R}^d$, rather they are concentrated around some low dimensional subspaces.
- These low dimensional structures in the data are the result of the inherent simplicity in the system producing the data.

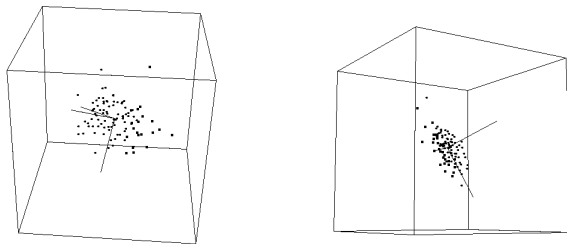
Intrinsic low dimensional structure

- A fact of hope is that observations are not randomly scattered in $\mathbb{R}^d$, rather they are concentrated around some low dimensional subspaces.
- These low dimensional structures in the data are the result of the inherent simplicity in the system producing the data.
- But the problem is that these low dimensional structures are unknown to us.

Intrinsic low dimensional structure

- A fact of hope is that observations are not randomly scattered in $\mathbb{R}^d$, rather they are concentrated around some low dimensional subspaces.
- These low dimensional structures in the data are the result of the inherent simplicity in the system producing the data.
- But the problem is that these low dimensional structures are unknown to us.
- Our objective today is to identify the low dimensional structures in the data.

Consider the 3-d scatterplot



- Rotating it we see that the points are all actually in a plane.
- So the intrinsic dimension is 2, and not 3.

How do we reduce the dimension?

- In the example above, we could identify the low dimensional structure by rotating the scatterplot.

How do we reduce the dimension?

- In the example above, we could identify the low dimensional structure by rotating the scatterplot.
- In case of further high dimension, we cannot make such plot.

How do we reduce the dimension?

- In the example above, we could identify the low dimensional structure by rotating the scatterplot.
- In case of further high dimension, we cannot make such plot.
- How can we identify the low dimensional structures?

How do we reduce the dimension?

- In the example above, we could identify the low dimensional structure by rotating the scatterplot.
- In case of further high dimension, we cannot make such plot.
- How can we identify the low dimensional structures?
- This is the main point of concern of all **dimension reduction techniques**.

Both PCA and FA reduce dimensions

- Both Principal component analysis and Factor Analysis reveal the low dimensional structure in the data.

Both PCA and FA reduce dimensions

- Both Principal component analysis and Factor Analysis reveal the low dimensional structure in the data.
- But they are not same.

Both PCA and FA reduce dimensions

- Both Principal component analysis and Factor Analysis reveal the low dimensional structure in the data.
- But they are not same.
- PCA reveals low dimensional structures derived from the original variables.

Both PCA and FA reduce dimensions

- Both Principal component analysis and Factor Analysis reveal the low dimensional structure in the data.
- But they are not same.
- PCA reveals low dimensional structures derived from the original variables.
- Factor Analysis looks back at history: tells us the story how the data was generated.

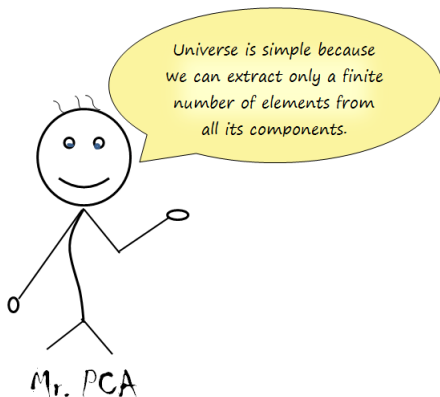
PCA vs Factor Analysis

- Question: How complex is the Universe?



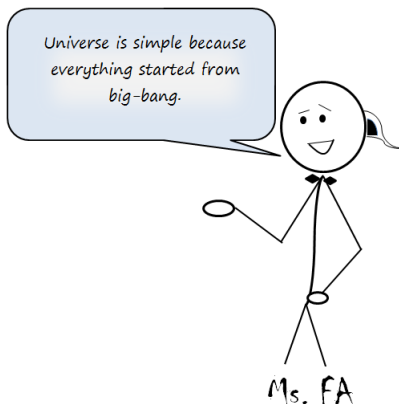
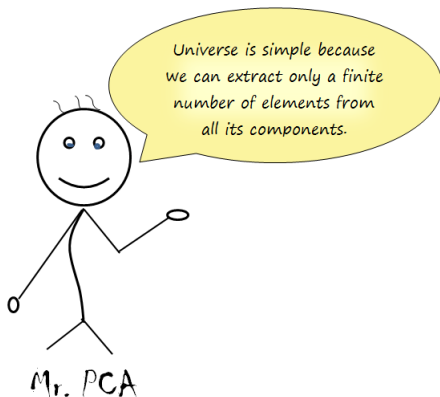
PCA vs Factor Analysis

- Question: How complex is the Universe?



PCA vs Factor Analysis

- Question: How complex is the Universe?



Taking Linear Projections

- We shall focus only on **linear dimensionality reduction**, in which, given high-dimensional data points $x_t \in \mathbb{R}^d$, we aim to derive lower dimensional “useful” representations $y_t \in \mathbb{R}^k$ as

$$y_t = Lx_t \text{ where } L = \begin{pmatrix} \ell_1^T \\ \vdots \\ \ell_k^T \end{pmatrix} \text{ is a } k \times d \text{ matrix.}$$

Taking Linear Projections

- We shall focus only on **linear dimensionality reduction**, in which, given high-dimensional data points $x_t \in \mathbb{R}^d$, we aim to derive lower dimensional “useful” representations $y_t \in \mathbb{R}^k$ as

$$y_t = Lx_t \text{ where } L = \begin{pmatrix} \ell_1^T \\ \vdots \\ \ell_k^T \end{pmatrix} \text{ is a } k \times d \text{ matrix.}$$

- But how do we construct such a matrix L ?

Taking Linear Projections

- We shall focus only on **linear dimensionality reduction**, in which, given high-dimensional data points $x_t \in \mathbb{R}^d$, we aim to derive lower dimensional “useful” representations $y_t \in \mathbb{R}^k$ as

$$y_t = Lx_t \text{ where } L = \begin{pmatrix} \ell_1^T \\ \vdots \\ \ell_k^T \end{pmatrix} \text{ is a } k \times d \text{ matrix.}$$

- But how do we construct such a matrix L ?
- As we shall see, PCA requires that we should choose the rows ℓ_i^T of L as mutually orthonormal rows.

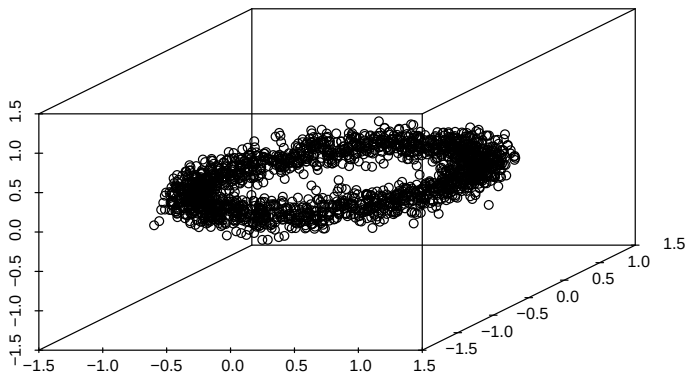
Taking Linear Projections

- We shall focus only on **linear dimensionality reduction**, in which, given high-dimensional data points $x_t \in \mathbb{R}^d$, we aim to derive lower dimensional “useful” representations $y_t \in \mathbb{R}^k$ as

$$y_t = Lx_t \text{ where } L = \begin{pmatrix} \ell_1^T \\ \vdots \\ \ell_k^T \end{pmatrix} \text{ is a } k \times d \text{ matrix.}$$

- But how do we construct such a matrix L ?
- As we shall see, PCA requires that we should choose the rows ℓ_i^T of L as mutually orthonormal rows.
- But will any L with mutually orthonormal rows work?

Example: Donut Dataset



Example (Continued)

- This is an artificial data set where we have 200 observations on 3 variables x , y and z .

Example (Continued)

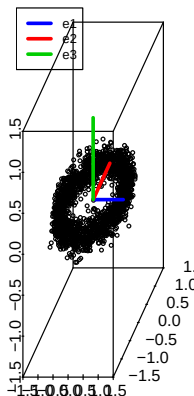
- This is an artificial data set where we have 200 observations on 3 variables x , y and z .
- The variable z is sampled from $N(0, 0.01)$ distribution and the variables x and y are generated as follows:

$$x = \cos(\theta) + \epsilon_1$$

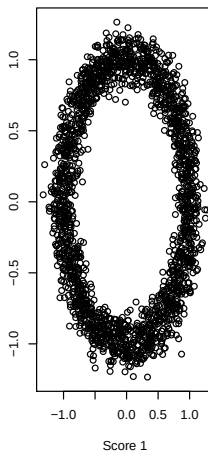
$$y = \sin(\theta) + \epsilon_2$$

where ϵ_1 and ϵ_2 are iid $N(0, 0.01)$ variables and θ is a constant which takes $n = 200$ equidistant values θ_i 's between 0 and 2π for the generation of the data (x_i, y_i) .

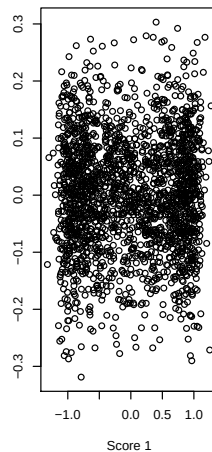
Projection onto orthonormal vectors



Scores onto e_1 and e_2



Scores onto e_1 and e_3



What do we get?

- The figure shows that not all linear projections are equal!

What do we get?

- The figure shows that not all linear projections are equal!
- So a natural question is: What makes a good one?

What do we get?

- The figure shows that not all linear projections are equal!
- So a natural question is: What makes a good one?
- The answer to this question is principal component analysis.

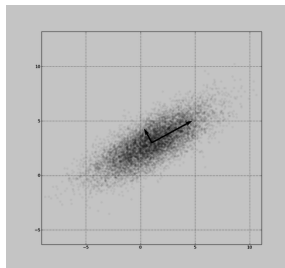
What do we get?

- The figure shows that not all linear projections are equal!
- So a natural question is: What makes a good one?
- The answer to this question is principal component analysis.
- In PCA , we just not only take linear projection onto orthonormal directions but also the variance along those directions is maximum.

- PCA finds an **orthonormal** basis (just a set of mutually perpendicular Cartesian axes, with origin at the centre of the data cloud).

- PCA finds an **orthonormal** basis (just a set of mutually perpendicular Cartesian axes, with origin at the centre of the data cloud).
- We may think of PCA as choosing a new coordinate system for the data.

- PCA finds an **orthonormal** basis (just a set of mutually perpendicular Cartesian axes, with origin at the centre of the data cloud).
- We may think of PCA as choosing a new coordinate system for the data.
- The coordinate system has the additional feature that the spread of the data cloud along the first direction is the maximum, that along the second direction is the second maximum and so on.



Data Setup

We are given a data matrix X , having n observations (row vectors) and d features (column vectors).

The n observations are $x_1, x_2, \dots, x_n$ all belonging to $\mathbb{R}^d$

That is

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix}$$

Data Setup

Let $\bar{x}$ be the mean vector and S be the dispersion matrix based on the data matrix X

$$\text{Then } \bar{x} = \frac{1}{n} \sum_{k=1}^n x_k = \frac{1}{n} X^T \mathbf{1} \text{ and}$$

$$S = \frac{1}{n} \sum_{k=1}^n (x_k - \bar{x})(x_k - \bar{x})^T = \frac{1}{n} (X - \mathbf{1}\bar{x}^T)^T (X - \mathbf{1}\bar{x}^T)$$

Data Setup

Then we define the centered data matrix as $Y = X - \mathbf{1}\bar{x}^T$

This centered matrix has mean $\bar{y} = 0$ and dispersion matrix

$$S_Y = \frac{1}{n} Y^T Y = \frac{1}{n} (X - \mathbf{1}\bar{x}^T)^T (X - \mathbf{1}\bar{x}^T) = S$$

We shall work with this centered matrix $Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_n^T \end{pmatrix}$

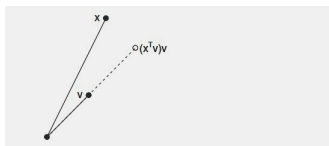
(Because it simplifies calculations)

Review: projections onto unit vectors

- The projection of $\mathbf{x} \in \mathbb{R}^d$ onto (the direction of) $\mathbf{v}$ is $(\mathbf{x}^T \mathbf{v})\mathbf{v}$.

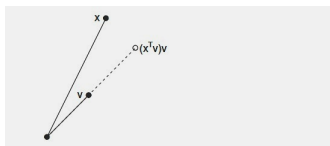
Review: projections onto unit vectors

- The projection of $\mathbf{x} \in \mathbb{R}^d$ onto (the direction of) $\mathbf{v}$ is $(\mathbf{x}^T \mathbf{v})\mathbf{v}$.
- This can be viewed as $c \times \mathbf{v}$, where c is a coefficient (or score along the direction of $\mathbf{v}$) $= \mathbf{x}^T \mathbf{v}$.



Review: projections onto unit vectors

- The projection of $\mathbf{x} \in \mathbb{R}^d$ onto (the direction of) $\mathbf{v}$ is $(\mathbf{x}^T \mathbf{v})\mathbf{v}$.
- This can be viewed as $c \times \mathbf{v}$, where c is a coefficient (or score along the direction of $\mathbf{v}$) $= \mathbf{x}^T \mathbf{v}$.



- Consider a matrix $X_{n \times d}$ and consider projecting each row $\mathbf{x}_i^T \in \mathbb{R}^d$ onto $\mathbf{v}$.

- The entries of $X\mathbf{v} = \begin{bmatrix} \mathbf{x}_1^T \mathbf{v} \\ \mathbf{x}_2^T \mathbf{v} \\ \vdots \\ \mathbf{x}_n^T \mathbf{v} \end{bmatrix} \in \mathbb{R}^n$ are the scores, and the rows of $X\mathbf{v}\mathbf{v}^T$ are the projected vectors.

Principal Components

- The first principal component direction of Y is the **unit vector** $\mathbf{v}_1 \in \mathbb{R}^d$ that maximizes the variance of $Y\mathbf{v}_1$ when compared to all other unit vectors.
- Now variance of $Y\mathbf{v}$ is

$$\frac{1}{n} \sum_{k=1}^n \left(y_k^T \mathbf{v} \right)^2$$

Principal Components

- The first principal component direction of Y is the **unit vector** $\mathbf{v}_1 \in \mathbb{R}^d$ that maximizes the variance of $Y\mathbf{v}_1$ when compared to all other unit vectors.
- Now variance of $Y\mathbf{v}$ is

$$\begin{aligned} & \frac{1}{n} \sum_{k=1}^n \left(y_k^T \mathbf{v} \right)^2 \\ &= \frac{1}{n} \sum_{k=1}^n \mathbf{v}^T (y_k y_k^T) \mathbf{v} \end{aligned}$$

Principal Components

- The first principal component direction of Y is the **unit vector** $\mathbf{v}_1 \in \mathbb{R}^d$ that maximizes the variance of $Y\mathbf{v}_1$ when compared to all other unit vectors.
- Now variance of $Y\mathbf{v}$ is

$$\begin{aligned} & \frac{1}{n} \sum_{k=1}^n \left(y_k^T \mathbf{v} \right)^2 \\ &= \frac{1}{n} \sum_{k=1}^n \mathbf{v}^T (y_k y_k^T) \mathbf{v} \\ &= \mathbf{v}^T \frac{1}{n} Y^T Y \mathbf{v} \end{aligned}$$

Principal Components

- The first principal component direction of Y is the **unit vector** $\mathbf{v}_1 \in \mathbb{R}^d$ that maximizes the variance of $Y\mathbf{v}_1$ when compared to all other unit vectors.
- Now variance of $Y\mathbf{v}$ is

$$\begin{aligned} & \frac{1}{n} \sum_{k=1}^n \left(y_k^T \mathbf{v} \right)^2 \\ &= \frac{1}{n} \sum_{k=1}^n \mathbf{v}^T (y_k y_k^T) \mathbf{v} \\ &= \mathbf{v}^T \frac{1}{n} Y^T Y \mathbf{v} \\ &= \mathbf{v}^T S \mathbf{v} \end{aligned}$$

Principal Components

- Thus

$$\mathbf{v}_1 = \underset{\mathbf{v}: \|\mathbf{v}\|_2=1}{\operatorname{argmax}} \mathbf{v}^T S \mathbf{v}$$

Principal Components

- Thus

$$\mathbf{v}_1 = \underset{\mathbf{v}: \|\mathbf{v}\|_2=1}{\operatorname{argmax}} \mathbf{v}^T S \mathbf{v}$$

- It can be shown that $\mathbf{v}_1$ is the eigenvector corresponding to the **largest** eigenvalue λ_1 of S .

Principal Components

- Thus

$$\mathbf{v}_1 = \underset{\mathbf{v}: \|\mathbf{v}\|_2=1}{\operatorname{argmax}} \mathbf{v}^T S \mathbf{v}$$

- It can be shown that $\mathbf{v}_1$ is the eigenvector corresponding to the **largest** eigenvalue λ_1 of S .
- Given the first principal component direction $\mathbf{v}_1$, we define the second principal component direction $\mathbf{v}_2$ to be the unit vector, **orthogonal** to $\mathbf{v}_1$ (i.e. $\mathbf{v}_2^T \mathbf{v}_1 = 0$), such that $X\mathbf{v}_2$ have maximal sample variance over all unit vectors orthogonal to $\mathbf{v}_1$.

Principal Components (Contd.)

- Thus

$$\mathbf{v}_2 = \underset{\mathbf{v}: \|\mathbf{v}\|_{l_2}=1, \mathbf{v}^T \mathbf{v}_1=0}{\operatorname{argmax}} \quad \mathbf{v}^T S \mathbf{v}$$

Principal Components (Contd.)

- Thus

$$\mathbf{v}_2 = \underset{\mathbf{v}: \|\mathbf{v}\|_{l_2}=1, \mathbf{v}^T \mathbf{v}_1=0}{\operatorname{argmax}} \quad \mathbf{v}^T S \mathbf{v}$$

- It can be shown (will show) that $\mathbf{v}_2$ is the eigenvector corresponding to the **second largest** eigenvalue λ_2 of S

Principal Components (Contd.)

- Thus

$$\mathbf{v}_2 = \underset{\mathbf{v}: \|\mathbf{v}\|_{l_2}=1, \mathbf{v}^T \mathbf{v}_1=0}{\operatorname{argmax}} \mathbf{v}^T S \mathbf{v}$$

- It can be shown (will show) that $\mathbf{v}_2$ is the eigenvector corresponding to the **second largest** eigenvalue λ_2 of S
- Proceeding in this way, the k^{th} principal component $\mathbf{v}_k$ is the eigenvector corresponding to the k^{th} **largest** eigenvalue λ_k of S

Why Orthogonal ?

- Because we've already explained the variance in Y along $\mathbf{v}_1$,
- Now we want to look at variance in a different direction.
- Any direction not orthogonal to $\mathbf{v}_1$ would necessarily have some overlap with $\mathbf{v}_1$, i.e., it would create some redundancy in explaining the variance in Y



Also, it makes the math easier !!!

Scores

- The vector $Y\mathbf{v}_1 \in \mathbb{R}^n$ is called the first principal component score of Y .

Scores

- The vector $Y\mathbf{v}_1 \in \mathbb{R}^n$ is called the first principal component score of Y .
- The vector $Y\mathbf{v}_2 \in \mathbb{R}^n$ is called the second principal component score of Y and so on.

Scores

- The vector $Y\mathbf{v}_1 \in \mathbb{R}^n$ is called the first principal component score of Y .
- The vector $Y\mathbf{v}_2 \in \mathbb{R}^n$ is called the second principal component score of Y and so on.
- These are the projections of the data points along the principal components.

Scores

- The vector $Y\mathbf{v}_1 \in \mathbb{R}^n$ is called the first principal component score of Y .
- The vector $Y\mathbf{v}_2 \in \mathbb{R}^n$ is called the second principal component score of Y and so on.
- These are the projections of the data points along the principal components.
- We have already mentioned that the principal components may be viewed as a new reference frame.

Scores

- The vector $Y\mathbf{v}_1 \in \mathbb{R}^n$ is called the first principal component score of Y .
- The vector $Y\mathbf{v}_2 \in \mathbb{R}^n$ is called the second principal component score of Y and so on.
- These are the projections of the data points along the principal components.
- We have already mentioned that the principal components may be viewed as a new reference frame.
- Then the scores are the coordinates of the points w.r.t. this frame.

Variance Explained

- The amount of variance explained by the first principal component is $\frac{1}{n}(Y\mathbf{v}_1)^T(Y\mathbf{v}_1)$

Variance Explained

- The amount of variance explained by the first principal component is $\frac{1}{n}(Y\mathbf{v}_1)^T(Y\mathbf{v}_1)$
- We can show that this variance is equal to the **largest** eigenvalue λ_1 of S .

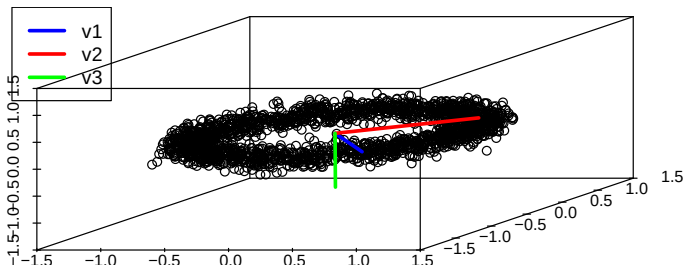
Variance Explained

- The amount of variance explained by the first principal component is $\frac{1}{n}(Y\mathbf{v}_1)^T(Y\mathbf{v}_1)$
- We can show that this variance is equal to the **largest** eigenvalue λ_1 of S .
- Proceeding in this way we get that amount of variance explained by k^{th} principal component is $\frac{1}{n}(Y\mathbf{v}_k)^T(Y\mathbf{v}_k)$, which is the k^{th} largest eigenvalue λ_k of S .

Donut Example revisited

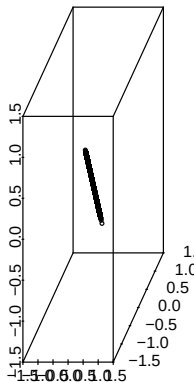
- Let us plot the principal component directions in the data cloud. Here v_1 , v_2 , v_3 are the unit vectors or in other words the principal component directions.

Principal component directions

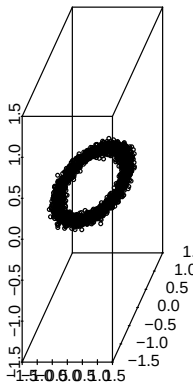


Projections of the data

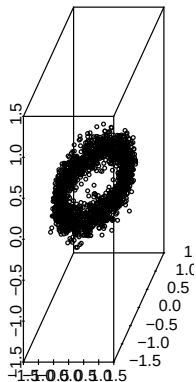
Projection onto v1



Projection onto v1,v2



Projection onto v1,v2,v3



What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.

What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.
- Further the second plot approximates the original data cloud to a very good extent.

What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.
- Further the second plot approximates the original data cloud to a very good extent.
- Hence we can easily approximate the original data by the projection onto v_1, v_2 .

What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.
- Further the second plot approximates the original data cloud to a very good extent.
- Hence we can easily approximate the original data by the projection onto v_1, v_2 .
- The third plot looks exactly the same as the original data. This is not a coincidence.

What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.
- Further the second plot approximates the original data cloud to a very good extent.
- Hence we can easily approximate the original data by the projection onto v_1, v_2 .
- The third plot looks exactly the same as the original data. This is not a coincidence.
- If we choose all the principal component directions, then there is no loss of information and no reduction in dimension.

What do we get?

- The first plot is a very crude approximation of the original data and cannot capture the original shape in the cloud.
- Further the second plot approximates the original data cloud to a very good extent.
- Hence we can easily approximate the original data by the projection onto v_1, v_2 .
- The third plot looks exactly the same as the original data. This is not a coincidence.
- If we choose all the principal component directions, then there is no loss of information and no reduction in dimension.
- The resultant projections are just a rotation of the original dataframe.

Dimension Reduction again

- Suppose that the data really are k -dimensional.

Dimension Reduction again

- Suppose that the data really are k -dimensional.
- Then S will have only k positive eigenvalues, and $d - k$ zero eigenvalues.

Dimension Reduction again

- Suppose that the data really are k -dimensional.
- Then S will have only k positive eigenvalues, and $d - k$ zero eigenvalues.
- If the data fall near a k -dimensional subspace, then $d - k$ of the eigenvalues will be nearly zero.

Dimension Reduction again

- Suppose that the data really are k -dimensional.
- Then S will have only k positive eigenvalues, and $d - k$ zero eigenvalues.
- If the data fall near a k -dimensional subspace, then $d - k$ of the eigenvalues will be nearly zero.
- In general, it may happen that the data really falls in a $m(< d)$ dimensional subspace and possibly near a $k(< m)$ dimensional subspace.

Dimension Reduction again

- Suppose that the data really are k -dimensional.
- Then S will have only k positive eigenvalues, and $d - k$ zero eigenvalues.
- If the data fall near a k -dimensional subspace, then $d - k$ of the eigenvalues will be nearly zero.
- In general, it may happen that the data really falls in a $m(< d)$ dimensional subspace and possibly near a $k(< m)$ dimensional subspace.
- In that case, we immediately discard the $d - m$ zero eigenvalues and search for further possible dimension reduction by checking whether additional $m - k$ eigenvalues can be found close to zero.

Case where $d > n$

- When $n < d$, PCA will surely be effective in reducing the dimension of the data.

Case where $d > n$

- When $n < d$, PCA will surely be effective in reducing the dimension of the data.
- Any two points define a line, and three points define a plane, etc.,

Case where $d > n$

- When $n < d$, PCA will surely be effective in reducing the dimension of the data.
- Any two points define a line, and three points define a plane, etc.,
- So if there are fewer data points than variables, it is necessarily true that they fall on a low-dimensional subspace.

Case where $d > n$

- When $n < d$, PCA will surely be effective in reducing the dimension of the data.
- Any two points define a line, and three points define a plane, etc.,
- So if there are fewer data points than variables, it is necessarily true that they fall on a low-dimensional subspace.
- This means that some of the eigenvalues of S will be exactly zero so that we can have a immediate reduction of dimension.

Case where $d > n$

- When $n < d$, PCA will surely be effective in reducing the dimension of the data.
- Any two points define a line, and three points define a plane, etc.,
- So if there are fewer data points than variables, it is necessarily true that they fall on a low-dimensional subspace.
- This means that some of the eigenvalues of S will be exactly zero so that we can have an immediate reduction of dimension.
- The point is we still need to search for additional small eigenvalues to check if we can further reduce the dimension.

How do we choose k ?

- In any case, we can define the R^2 of the projection as the fraction of the original variance kept by the image vectors,

$$R^2 = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

How do we choose k ?

- In any case, we can define the R^2 of the projection as the fraction of the original variance kept by the image vectors,

$$R^2 = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

- This is just as the R^2 of a linear regression is the fraction of the original variance of the dependent variable kept by the fitted values.

How do we choose k ?

- In any case, we can define the R^2 of the projection as the fraction of the original variance kept by the image vectors,

$$R^2 = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

- This is just as the R^2 of a linear regression is the fraction of the original variance of the dependent variable kept by the fitted values.
- Usually, to get an R^2 of 1, we need to use all d principal components (unless there are some immediate variable dependency)

How do we choose k ?

- In any case, we can define the R^2 of the projection as the fraction of the original variance kept by the image vectors,

$$R^2 = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

- This is just as the R^2 of a linear regression is the fraction of the original variance of the dependent variable kept by the fitted values.
- Usually, to get an R^2 of 1, we need to use all d principal components (unless there are some immediate variable dependency)
- How many principal components we should use depends on our data, and how big an R^2 we need.

How do we choose k ?

- In any case, we can define the R^2 of the projection as the fraction of the original variance kept by the image vectors,

$$R^2 = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

- This is just as the R^2 of a linear regression is the fraction of the original variance of the dependent variable kept by the fitted values.
- Usually, to get an R^2 of 1, we need to use all d principal components (unless there are some immediate variable dependency)
- How many principal components we should use depends on our data, and how big an R^2 we need.
- Generally k is chosen such that R^2 is close to 1 (say more than 0.9).

Screeplots

- People sometimes like to make plots of the eigenvalues, in decreasing order.

Screeplots

- People sometimes like to make plots of the eigenvalues, in decreasing order.
- Ideally, we start with a few big eigenvalues, and then there is a clear drop-off to a remainder of small, comparatively negligible eigenvalues.

Screeplots

- People sometimes like to make plots of the eigenvalues, in decreasing order.
- Ideally, we start with a few big eigenvalues, and then there is a clear drop-off to a remainder of small, comparatively negligible eigenvalues.
- These diagrams are called scree plots.

Screeplots

- People sometimes like to make plots of the eigenvalues, in decreasing order.
- Ideally, we start with a few big eigenvalues, and then there is a clear drop-off to a remainder of small, comparatively negligible eigenvalues.
- These diagrams are called scree plots.
- Some people make similar plots, but show $1 - R^2$ or R^2 versus the number of components, rather than the individual eigenvalues.

Screeplots

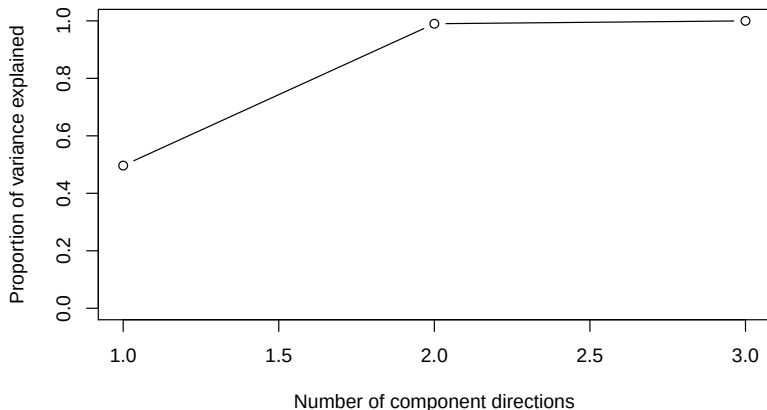
- People sometimes like to make plots of the eigenvalues, in decreasing order.
- Ideally, we start with a few big eigenvalues, and then there is a clear drop-off to a remainder of small, comparatively negligible eigenvalues.
- These diagrams are called scree plots.
- Some people make similar plots, but show $1 - R^2$ or R^2 versus the number of components, rather than the individual eigenvalues.
- Folklore recommends find the “base of the cliff” or “elbow” in the plot, the place where the number eigenvalues decrease dramatically and then level off to the right, and then retaining that number of components.

Screeplots

- People sometimes like to make plots of the eigenvalues, in decreasing order.
- Ideally, we start with a few big eigenvalues, and then there is a clear drop-off to a remainder of small, comparatively negligible eigenvalues.
- These diagrams are called scree plots.
- Some people make similar plots, but show $1 - R^2$ or R^2 versus the number of components, rather than the individual eigenvalues.
- Folklore recommends find the “base of the cliff” or “elbow” in the plot, the place where the number eigenvalues decrease dramatically and then level off to the right, and then retaining that number of components.
- But this suggestion is nothing more than intuition, and offers no recommendation for what to do when there is no clear cliff or elbow in the scree plot.

Donut Example once again

- We can plot the cumulative variance explained by the principal components.



From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.

From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.
- Since the original observations are d dimensional, we have d principal components each of which are a vector of length d .

From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.
- Since the original observations are d dimensional, we have d principal components each of which are a vector of length d .
- Let us stack these components in a matrix $L_{d \times d}$.

From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.
- Since the original observations are d dimensional, we have d principal components each of which are a vector of length d .
- Let us stack these components in a matrix $L_{d \times d}$.
- Finally, each data point has a projection on to each principal component, which we collect into an $n \times d$ matrix F .

From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.
- Since the original observations are d dimensional, we have d principal components each of which are a vector of length d .
- Let us stack these components in a matrix $L_{d \times d}$.
- Finally, each data point has a projection on to each principal component, which we collect into an $n \times d$ matrix F .
- Then we get an exact equation

$$X = FL$$

From PCA to Factor Analysis

- Suppose our data matrix is $X_{n \times d}$.
- Since the original observations are d dimensional, we have d principal components each of which are a vector of length d .
- Let us stack these components in a matrix $L_{d \times d}$.
- Finally, each data point has a projection on to each principal component, which we collect into an $n \times d$ matrix F .
- Then we get an exact equation

$$X = FL$$

- This exact equation is basically useless since this is nothing but rotation of axes in the d dimensional space.

PCA to Factor Analysis (Contd.)

- If we keep only to $k < d$ largest principal components, that corresponds to dropping columns from F and rows from W .

PCA to Factor Analysis (Contd.)

- If we keep only to $k < d$ largest principal components, that corresponds to dropping columns from F and rows from W .
- Denoting the truncated matrices as F_k and L_k we get

$$X \approx F_k L_k.$$

PCA to Factor Analysis (Contd.)

- If we keep only to $k < d$ largest principal components, that corresponds to dropping columns from F and rows from W .
- Denoting the truncated matrices as F_k and L_k we get

$$X \approx F_k L_k.$$

- This is an approximate equation and it involves errors of approximation.

PCA to Factor Analysis (Contd.)

- If we keep only to $k < d$ largest principal components, that corresponds to dropping columns from F and rows from W .
- Denoting the truncated matrices as F_k and L_k we get

$$X \approx F_k L_k.$$

- This is an approximate equation and it involves errors of approximation.
- The error of approximation will get smaller as we increase k upto d .

PCA to Factor Analysis (Contd.)

- If we keep only to $k < d$ largest principal components, that corresponds to dropping columns from F and rows from W .
- Denoting the truncated matrices as F_k and L_k we get

$$X \approx F_k L_k.$$

- This is an approximate equation and it involves errors of approximation.
- The error of approximation will get smaller as we increase k upto d .
- Recall that in PCA we were satisfied with this approximation if it is upto a reasonable extent.

PCA to Factor Analysis (Contd.)

- We can also make the two sides match exactly by adding an error or residual term on the right

$$X = FL + \epsilon$$

where ϵ is a $n \times d$ matrix and F and L now stands for F_k and L_k .

PCA to Factor Analysis (Contd.)

- We can also make the two sides match exactly by adding an error or residual term on the right

$$X = FL + \epsilon$$

where ϵ is a $n \times d$ matrix and F and L now stands for F_k and L_k .

- Let us start treating the noise ϵ as random which has some distribution.

PCA to Factor Analysis (Contd.)

- We can also make the two sides match exactly by adding an error or residual term on the right

$$X = FL + \epsilon$$

where ϵ is a $n \times d$ matrix and F and L now stands for F_k and L_k .

- Let us start treating the noise ϵ as random which has some distribution.
- This is where we move from mere empirical data reduction to making assertions about the process that generated the data.

PCA to Factor Analysis (Contd.)

- We say that we have k **factors**.

PCA to Factor Analysis (Contd.)

- We say that we have k **factors**.
- F is the matrix of factor scores (rather than projections onto principal components).

PCA to Factor Analysis (Contd.)

- We say that we have k **factors**.
- F is the matrix of factor scores (rather than projections onto principal components).
- L is the matrix of factor loadings.

PCA to Factor Analysis (Contd.)

- We say that we have k **factors**.
- F is the matrix of factor scores (rather than projections onto principal components).
- L is the matrix of factor loadings.
- The variables in X are called observable or manifest variables, those in F are **hidden** or **latent**. (Technically ϵ is also latent.)

Example

- Consider a market study regarding customers of a brand of car.
- Data on **14 variables** are obtained.
- The customers were asked to rate their preferences in a scale (from 1 to 7) for each of these variables.

Variables

- 1 Low cost of repairs
- 2 Comes in a variety of colors
- 3 Roomy Interior
- 4 Good Gas Mileage
- 5 Good Handling
- 6 Modern Looking
- 7 High Resale Value
- 8 Comfortable
- 9 Large Engine
- 10 Sleek Appearance
- 11 Easy to drive
- 12 Eye Catching
- 13 Large Trunk Space
- 14 Easy to park

A close look at the variables

- ① Low cost of repairs
- ② Comes in a variety of colors
- ③ Roomy Interior
- ④ Good Gas Mileage
- ⑤ **Good Handling**
- ⑥ Modern Looking
- ⑦ High Resale Value
- ⑧ Comfortable
- ⑨ Large Engine
- ⑩ Sleek Appearance
- ⑪ **Easy to drive**
- ⑫ Eye Catching
- ⑬ Large Trunk Space
- ⑭ **Easy to park**

Variables 5, 11, 14
represents the customer
choice about **driving**.

A close look at the variables

- 1 Low cost of repairs
- 2 **Comes in a variety of colors**
- 3 Roomy Interior
- 4 Good Gas Mileage
- 5 Good Handling
- 6 **Modern Looking**
- 7 High Resale Value
- 8 Comfortable
- 9 Large Engine
- 10 **Sleek Appearance**
- 11 Easy to drive
- 12 **Eye Catching**
- 13 Large Trunk Space
- 14 Easy to park

Variables 2,6,10,12
represents the customer
choice about **looks**.

A close look at the variables

- ① **Low cost of repairs**
- ② Comes in a variety of colors
- ③ Roomy Interior
- ④ **Good Gas Mileage**
- ⑤ Good Handling
- ⑥ Modern Looking
- ⑦ **High Resale Value**
- ⑧ Comfortable
- ⑨ **Large Engine**
- ⑩ Sleek Appearance
- ⑪ Easy to drive
- ⑫ Eye Catching
- ⑬ Large Trunk Space
- ⑭ Easy to park

Variables 1,4,7,9
represents the customer
choice about **economy**.

A close look at the variables

- ① Low cost of repairs
- ② Comes in a variety of colors
- ③ **Roomy Interior**
- ④ Good Gas Mileage
- ⑤ Good Handling
- ⑥ Modern Looking
- ⑦ High Resale Value
- ⑧ **Comfortable**
- ⑨ Large Engine
- ⑩ Sleek Appearance
- ⑪ Easy to drive
- ⑫ Eye Catching
- ⑬ **Large Trunk Space**
- ⑭ Easy to park

Variables 3,8,13 represents the customer choice about **comfortability**.

What do we get?

- Although we have 14 variables, many of the **variables are similar**.

What do we get?

- Although we have 14 variables, many of the **variables are similar**.
- In a nutshell the data can be described only by the variables **Ease of Driving, Looks, Comfort, Economy**.

What do we get?

- Although we have 14 variables, many of the **variables are similar**.
- In a nutshell the data can be described only by the variables **Ease of Driving, Looks, Comfort, Economy**.
- Thus instead of 14 observed variables we can only work with these **4 underlying variables**.

What do we get?

- Although we have 14 variables, many of the **variables are similar**.
- In a nutshell the data can be described only by the variables **Ease of Driving, Looks, Comfort, Economy**.
- Thus instead of 14 observed variables we can only work with these **4 underlying variables**.
- These underlying variables are called **factors**.

What do we get?

- Although we have 14 variables, many of the **variables are similar**.
- In a nutshell the data can be described only by the variables **Ease of Driving, Looks, Comfort, Economy**.
- Thus instead of 14 observed variables we can only work with these **4 underlying variables**.
- These underlying variables are called **factors**.
- Representing variables in terms of these unknown factors is called **factor analysis**.

Orthogonal Factor model

- It is to be noted that this model can also be stated in terms of the data matrix as

$$X_{n \times d} = F_{n \times k} L_{k \times d} + \epsilon_{n \times d}$$

- Before we can actually do much with this model, we need to say more about the distributions of these random variables.

Orthogonal Factor model

- It is to be noted that this model can also be stated in terms of the data matrix as

$$X_{n \times d} = F_{n \times k} L_{k \times d} + \epsilon_{n \times d}$$

- Before we can actually do much with this model, we need to say more about the distributions of these random variables.
- All of the latent factors have mean zero and variance 1
- The factors are uncorrelated.
 - That is, $E(F) = 0$ and $E(FF^T) = I_k$.

- The noise terms all have mean zero
- The noise terms are uncorrelated.
 - That is, $E(\epsilon) = 0$ and $E(\epsilon\epsilon^T) = \psi = \text{diag}(\psi_1, \psi_2, \dots, \psi_d)$.

- The noise terms all have mean zero
- The noise terms are uncorrelated.
 - That is, $E(\epsilon) = 0$ and $E(\epsilon\epsilon^T) = \psi = \text{diag}(\psi_1, \psi_2, \dots, \psi_d)$.
- The noise terms are uncorrelated with the factor variables, that is, $E(\epsilon F^T) = 0$.
- The factors are uncorrelated across individuals (rows of F) (in addition to across variables (columns of F)).
- The noise terms are uncorrelated across individuals (in addition to across observable variables).

What do these assumptions imply?

- These assumptions in the factor model has a strong implication regarding the covariance structure of X as

$$\Sigma = L^T L + \psi$$

What do these assumptions imply?

- These assumptions in the factor model has a strong implication regarding the covariance structure of X as

$$\Sigma = L^T L + \psi$$

- Thus the variance of X_i can be split as

$$\text{Var}(X_i) = \underbrace{l_{i1}^2 + l_{i2}^2 + \dots + l_{ik}^2}_{\text{communality}} + \underbrace{\psi_i}_{\text{specific variance}}$$

What do these assumptions imply?

- These assumptions in the factor model has a strong implication regarding the covariance structure of X as

$$\Sigma = L^T L + \psi$$

- Thus the variance of X_i can be split as

$$\text{Var}(X_i) = \underbrace{l_{i1}^2 + l_{i2}^2 + \dots + l_{ik}^2}_{\text{communality}} + \underbrace{\psi_i}_{\text{specific variance}}$$

- Thus the i^{th} communality is the sum of squares of loadings of i^{th} variable on all the k common factors.

PCA in R

- There are two R functions for doing PCA, princomp and prcomp, which differ in how they do the actual calculation.

PCA in R

- There are two R functions for doing PCA, princomp and prcomp, which differ in how they do the actual calculation.
- The latter is generally more robust.

PCA in R

- There are two R functions for doing PCA, princomp and prcomp, which differ in how they do the actual calculation.
- The latter is generally more robust.
- Alternatively we can also perform eigen analysis of the dispersion matrix.

Example: state.x77

- R contains a built-in data file, **state.x77**, with facts and figures for the various states of the USA as of about 1977.

Example: state.x77

- R contains a built-in data file, **state.x77**, with facts and figures for the various states of the USA as of about 1977.
- The data set contains population, per-capita income, the adult illiteracy rate, life expectancy, the homicide rate, the proportion of adults with at least a high-school education, the number of days of frost a year, and the state's area.

Example: state.x77

- R contains a built-in data file, **state.x77**, with facts and figures for the various states of the USA as of about 1977.
- The data set contains population, per-capita income, the adult illiteracy rate, life expectancy, the homicide rate, the proportion of adults with at least a high-school education, the number of days of frost a year, and the state's area.
- Since the variables all have different, incomparable scales, it's not a bad idea to scale them to unit variance before finding the components.

R code

```
state.pca <- prcomp(state.x77, scale. = TRUE)
```

- We can now extract the loadings or weight matrix from the state.pca object.
- For comprehensibility we'll just show the first two components.

Loading matrix

```
signif(state.pca$rotation[, 1:2], 2)
```

	PC1	PC2
Population	0.130	0.410
Income	-0.300	0.520
Illiteracy	0.470	0.053
Life Exp	-0.410	-0.082
Murder	0.440	0.310
HS Grad	-0.420	0.300
Frost	-0.360	-0.150
Area	-0.033	0.590

What do we get?

- The first component aligns with illiteracy, murder, and (more weakly) population; it's negatively aligned with high school graduation, life expectancy, cold weather, income, and (very weakly) the area of the state.

What do we get?

- The first component aligns with illiteracy, murder, and (more weakly) population; it's negatively aligned with high school graduation, life expectancy, cold weather, income, and (very weakly) the area of the state.
- The second component is positively aligned with area, income, population, high school graduation and murder, and negatively aligned, weakly, with cold weather and life expectancy.

What do we get?

- The first component aligns with illiteracy, murder, and (more weakly) population; it's negatively aligned with high school graduation, life expectancy, cold weather, income, and (very weakly) the area of the state.
- The second component is positively aligned with area, income, population, high school graduation and murder, and negatively aligned, weakly, with cold weather and life expectancy.
- The first component thus separates short-lived, violent, ill-educated, poor warm states from those with the opposite qualities.

What do we get?

- The first component aligns with illiteracy, murder, and (more weakly) population; it's negatively aligned with high school graduation, life expectancy, cold weather, income, and (very weakly) the area of the state.
- The second component is positively aligned with area, income, population, high school graduation and murder, and negatively aligned, weakly, with cold weather and life expectancy.
- The first component thus separates short-lived, violent, ill-educated, poor warm states from those with the opposite qualities.
- The second component separates big, rich, educated, violent states from those which are small (in land or people), poor, less educated, and less violent.

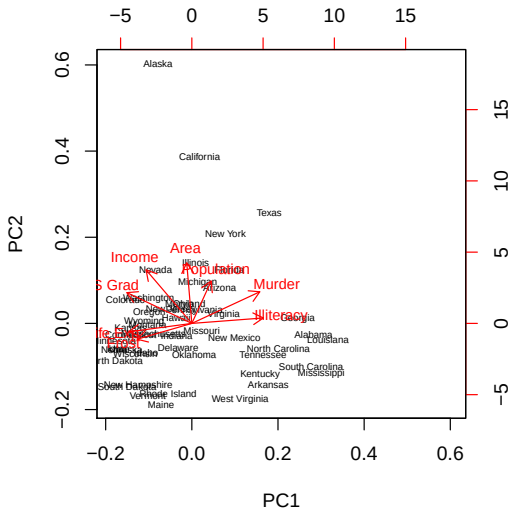
Biplot I

- To check this interpretation, we can use a useful tool called a biplot, which plots the data, along with the projections of the original variables, on to the first two components.

Biplot II

```
biplot(state.pca, cex = c(0.5, 0.75))
```

Biplot III



Interpretation

- Since each data point has a geographic location, we can make a map, where the sizes of the symbols for each state vary with their projection on to the first principal component.

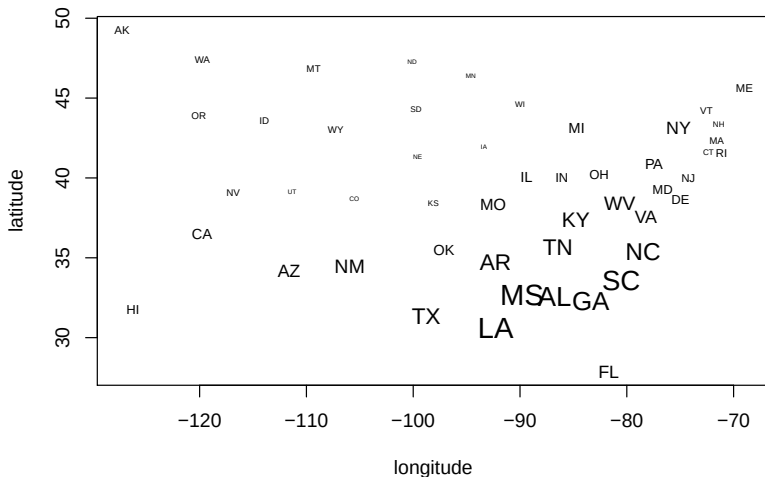
Interpretation

- Since each data point has a geographic location, we can make a map, where the sizes of the symbols for each state vary with their projection on to the first principal component.
- This suggests that the component is something we might call “southernness” — more precisely, the contrast between the South and the rest of the nation.

US states plotted in their geographic locations I

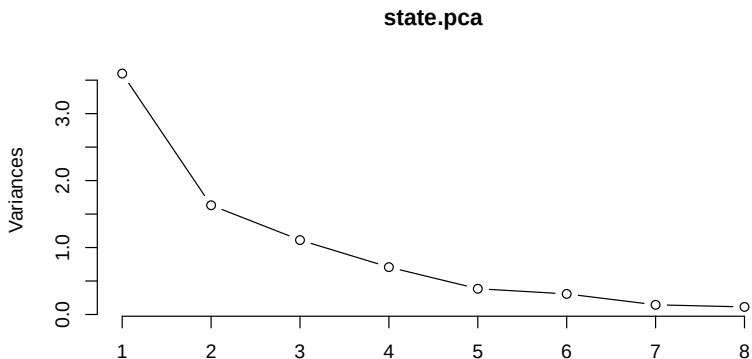
```
plot.states<-function(sizes,min.size=0.4,max.size=2,...)
{
  plot(state.center,type="n",...)
  out.range=max.size-min.size
  in.range=max(sizes)-min(sizes)
  scaled.sizes=out.range*((sizes-min(sizes))/in.range)
  text(state.center,state.abb, cex = scaled.sizes + min.size)
  invisible(scaled.sizes)
}
plot.states(state.pca$x[, 1], min.size = 0.3,
max.size=1.5,xlab="longitude",ylab="latitude")
```


US states plotted in their geographic locations II



Screplot

```
plot(state.pca, type = "l")
```



Factor Analysis in R

- R has an inbuilt function **factanal()** which performs factor analysis by maximum likelihood method.
- We look back at the data set containing the properties of the US states around 1977.
- Let us subject this dataset to factor analysis and see what latent structures we can discover.

Let's begin in R

We begin with one factor, using the base R function `factanal`.

```
state.fal <- factanal(state.x77,factors=1,scores="regression")
state.fal
```

Call:

```
factanal(x = state.x77, factors = 1, scores = "regression")
```

Uniquenesses:

Population	Income	Illiteracy	Life Exp	Murder	HS Grad
0.957	0.791	0.235	0.437	0.308	0.496
Frost	Area				
0.600	0.998				

Loadings:

	Factor1
Population	-0.208
Income	0.458
Illiteracy	-0.875
Life Exp	0.750
Murder	-0.832
HS Grad	0.710
Frost	0.632
Area	

	Factor1
SS loadings	3.178
Proportion Var	0.397

Test of the hypothesis that 1 factor is sufficient.

What do we get?

- The uniqueness in the output here tells us what fraction of the variance in each observable comes from its own noise which is nothing but the diagonal entries in $\hat{\psi}$.

What do we get?

- The uniqueness in the output here tells us what fraction of the variance in each observable comes from its own noise which is nothing but the diagonal entries in $\hat{\psi}$.
- It also gives us the factor loadings, i.e., the rows of $\hat{L}$.

What do we get?

- The uniqueness in the output here tells us what fraction of the variance in each observable comes from its own noise which is nothing but the diagonal entries in $\hat{\psi}$.
- It also gives us the factor loadings, i.e., the rows of $\hat{L}$.
- Note that currently there's only one loading vector, since we set the numebr of factors (k) to be 1 by the argument `factors= 1`.

What do we get?

- The uniqueness in the output here tells us what fraction of the variance in each observable comes from its own noise which is nothing but the diagonal entries in $\hat{\psi}$.
- It also gives us the factor loadings, i.e., the rows of $\hat{L}$.
- Note that currently there's only one loading vector, since we set the numebr of factors (k) to be 1 by the argument `factors= 1`.
- As a courtesy, the default printing method for the loadings leaves blanks where the loadings would be very small (here, for Area).

What do we get?

- The uniqueness in the output here tells us what fraction of the variance in each observable comes from its own noise which is nothing but the diagonal entries in $\hat{\psi}$.
- It also gives us the factor loadings, i.e., the rows of $\hat{L}$.
- Note that currently there's only one loading vector, since we set the numebr of factors (k) to be 1 by the argument `factors= 1`.
- As a courtesy, the default printing method for the loadings leaves blanks where the loadings would be very small (here, for Area).
- Finally the last option picks between different methods of estimating the factor scores.

More than one factor

- Of course, why use just one factor?

More than one factor

- Of course, why use just one factor?
- Now given the number of observables, we can fit up to four factors before the problem becomes totally unidentified and factanal refuses to work.

More than one factor

- Of course, why use just one factor?
- Now given the number of observables, we can fit up to four factors before the problem becomes totally unidentified and factanal refuses to work.
- That function factanal automatically runs the likelihood ratio test every time it fits a model, assuming Gaussian distributions for the observables.

More than one factor

- Of course, why use just one factor?
- Now given the number of observables, we can fit up to four factors before the problem becomes totally unidentified and `factanal` refuses to work.
- That function `factanal` automatically runs the likelihood ratio test every time it fits a model, assuming Gaussian distributions for the observables.
- This can work reasonably well for non-Gaussian distributions if they're not too non- Gaussian, especially if n is much larger than the number of parameters

More than one factor

- Of course, why use just one factor?
- Now given the number of observables, we can fit up to four factors before the problem becomes totally unidentified and factanal refuses to work.
- That function factanal automatically runs the likelihood ratio test every time it fits a model, assuming Gaussian distributions for the observables.
- This can work reasonably well for non-Gaussian distributions if they're not too non- Gaussian, especially if n is much larger than the number of parameters
- Of course $n = 50$ is pretty modest. Still, we shall try it.

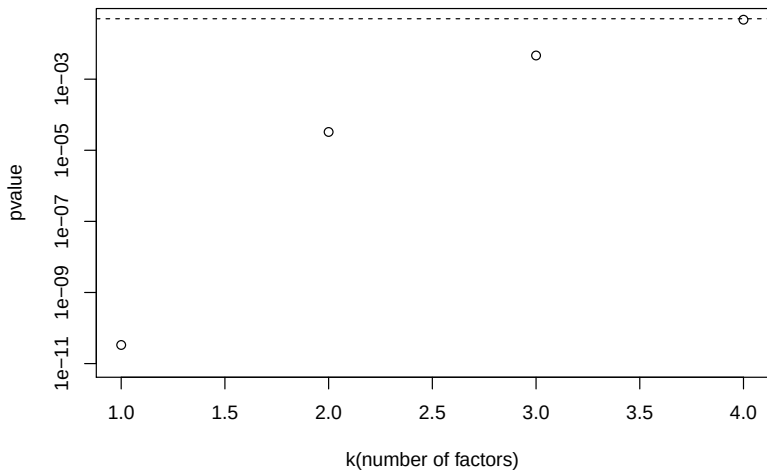
Which model is best? I

```
pvalues<-sapply(1:4,function(q){factanal(state.x77,factors=q)$PVAL})  
signif(pvalues,2)
```

```
objective objective objective objective  
3.3e-11 3.3e-05 4.6e-03 4.7e-02
```

```
plot(1:4,pvalues,xlab="k(number of factors)", ylab="pvalue",  
log="y",ylim=c(1e-11,0.04))  
abline(h=0.05,lty="dashed")
```

Which model is best? II



What do we get?

- None of the models has a p-value crossing the conventional 0.05 level.

What do we get?

- None of the models has a p-value crossing the conventional 0.05 level.
- This means all of them show systematic, detectable departures from what the data should look like if the factor model were true.

What do we get?

- None of the models has a p-value crossing the conventional 0.05 level.
- This means all of them show systematic, detectable departures from what the data should look like if the factor model were true.
- Still, the four factor model comes close.

What do we get?

- None of the models has a p-value crossing the conventional 0.05 level.
- This means all of them show systematic, detectable departures from what the data should look like if the factor model were true.
- Still, the four factor model comes close.
- In other words, given the data, if we need to explain the variables in terms of few underlying factors, the four factor model does the best.

Loading matrix

- Notice that the first factor's loadings do not stay the same when we add more factors, unlike the first principal component.

```
print(factanal(state.x77, factors=4)$loadings)
```

Loadings:

	Factor1	Factor2	Factor3	Factor4
Population				0.636
Income	0.313	0.281	0.561	0.189
Illiteracy	-0.466	-0.878		
Life Exp	0.891	0.191		
Murder	-0.792	-0.384	0.109	0.405
HS Grad	0.517	0.418	0.581	
Frost	0.128	0.679	0.105	-0.460
Area	-0.174		0.796	

	Factor1	Factor2	Factor3	Factor4
SS loadings	2.054	1.680	1.321	0.821
Proportion Var	0.257	0.210	0.165	0.103
Cumulative Var	0.257	0.467	0.632	0.734